Architetture dei Sistemi Distribuiti

Lab — HTTP Protocol & HTTP Redirect Load Balancing

Lezione Pratica

A.A. 2025–2026

Prof. Luca Vollero
Università Campus Bio-Medico di Roma

Handout per la sessione di laboratorio

Obiettivi della Lezione

Al termine di questa sessione pratica lo studente sarà in grado di:

1. Descrivere il modello request/response di HTTP e il ruolo di metodi, status code e header.
2. Leggere ed interpretare una transazione HTTP completa usando strumenti da riga di comando (curl, netcat).
3. Implementare un server HTTP minimale in Python utilizzando la libreria standard.
4. Comprendere il meccanismo di HTTP redirect (3xx) e la differenza architetturale tra 301 e 302.
5. Realizzare un load balancer L7 basato su redirect con strategia round-robin.
6. Osservare e misurare il comportamento della cache del browser di fronte a redirect permanenti vs temporanei.

Prerequisiti

- Python 3.8+ installato e funzionante
- Familiarità con il terminale (bash/PowerShell)
- Concetti base di networking: TCP, IP, porte, modello client-server
- Browser moderno con Developer Tools (Chrome consigliato)

File di Laboratorio

I file si trovano nel repository del corso alla path:

```
labs/lb_example/http/
```

File	Ruolo	Righe
`http_backend.py`	Server HTTP backend multi-endpoint	~335
`http_redirect.py`	Load balancer HTTP via redirect (301/302)	~440

Parte 1 — Il Protocollo HTTP

1.1 Che cos'è HTTP?

HTTP (**HyperText Transfer Protocol**) è il protocollo applicativo alla base del World Wide Web. Definito originariamente da Tim Berners-Lee nel 1991, è oggi il meccanismo dominante per la comunicazione tra client e server nelle architetture distribuite.

Le caratteristiche fondamentali di HTTP sono:

- **Request/Response:** il client invia una richiesta, il server produce una risposta. Ogni scambio è atomico e autocontenuto.
- **Stateless:** il server non mantiene stato tra una richiesta e la successiva. Ogni richiesta deve contenere tutte le informazioni necessarie per essere processata. Questo semplifica enormemente lo scaling orizzontale.
- **Text-based (in HTTP/1.1):** i messaggi sono leggibili dall'essere umano, il che facilita il debugging e l'apprendimento.
- **Estensibile:** gli header permettono di aggiungere metadati arbitrari senza modificare il protocollo.

> **Nota Architetturale**
> La natura stateless di HTTP è una scelta progettuale deliberata: rende il protocollo intrinsecamente scalabile. Un load balancer può inoltrare richieste successive dello stesso client a server diversi senza problemi. Lo stato, quando necessario, viene gestito a livello applicativo (cookie, session token, JWT).

1.2 Anatomia di una Richiesta HTTP

Una richiesta HTTP è composta da tre parti distinte, separate da CRLF (\r\n):

Struttura di una richiesta HTTP

```
GET /api/info?format=json HTTP/1.1      ← Request Line
Host: 127.0.0.1:8001                    ← Headers
User-Agent: curl/8.4.0
Accept: */*
                                         ← Riga vuota (fine headers)
[body opzionale per POST/PUT]             ← Body
```

Request Line — contiene tre elementi:

- **Metodo** (GET, POST, PUT, DELETE, ...): indica l'azione desiderata
- **Path** (/api/info?format=json): la risorsa richiesta, con eventuali query parameters
- **Versione** (HTTP/1.1): la versione del protocollo

1.3 I Metodi HTTP Principali

Metodo	Semantica	Safe?	Idempotent?
GET	Recupera una risorsa	Sì	Sì
POST	Crea una nuova risorsa / invia dati	No	No
PUT	Sostituisce interamente una risorsa	No	Sì
DELETE	Elimina una risorsa	No	Sì
PATCH	Modifica parziale di una risorsa	No	No
HEAD	Come GET ma senza body nella risposta	Sì	Sì
OPTIONS	Richiede i metodi supportati	Sì	Sì

Safe: il metodo non modifica lo stato del server. **Idempotent**: ripetere la stessa richiesta N volte produce lo stesso risultato della singola esecuzione. Queste proprietà sono definite nella RFC 7231 e hanno implicazioni dirette sul design di sistemi distribuiti: un load balancer può ripetere (retry) una richiesta GET sicura in caso di failure, ma non dovrebbe farlo per una POST.

1.4 Anatomia di una Risposta HTTP

Struttura di una risposta HTTP

```
HTTP/1.1 200 OK                          ← Status Line
Content-Type: application/json            ← Headers
Content-Length: 127
X-Served-By: backend-8001                ← Header custom
Cache-Control: no-store

{"server_port": 8001, ...}               ← Body
```

1.5 I Codici di Stato HTTP

I codici di stato sono raggruppati in cinque classi, ciascuna con una semantica precisa:

Classe	Significato	Esempi Chiave
1xx	Informational — risposta provvisoria	100 Continue, 101 Switching Protocols
2xx	Success — richiesta completata	200 OK, 201 Created, 204 No Content
3xx	Redirection — azione ulteriore necessaria	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Error — richiesta errata	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Error — errore lato server	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Focus: la famiglia 3xx

I codici 3xx sono centrali per questa lezione. Il meccanismo di redirect è usato ovunque nel web moderno: URL shortener, CDN, load balancing, migrazione di dominio, HTTPS enforcement. L'header `Location` nella risposta indica al client dove reindirizzare la richiesta.

1.6 Gli Header HTTP più Importanti

Header	Direzione	Scopo
`Host`	Request	Indica il server di destinazione (obbligatorio in HTTP/1.1)
`Content-Type`	Entrambe	Tipo MIME del body (es. application/json, text/html)
`Content-Length`	Entrambe	Dimensione del body in byte
`Location`	Response	URL di destinazione per i redirect (3xx)
`Cache-Control`	Entrambe	Direttive di caching (no-store, max-age, ...)
`User-Agent`	Request	Identifica il client (browser, curl, bot, ...)
`Referer`	Request	URL della pagina da cui proviene la richiesta

Oltre agli header standard, è pratica comune definire header custom con il prefisso `X-`. Nel nostro laboratorio, il backend usa `X-Served-By` e il load balancer usa `X-LB-Mode`, `X-LB-Backend` e `X-LB-Request-Count` per il debugging.

Parte 2 — Esercitazione: HTTP "a Mani Nude"

Prima di esaminare il codice Python, tocchiamo con mano il protocollo inviando e ricevendo messaggi HTTP grezzi.

2.1 Osservare HTTP con curl

Il flag `-v` (verbose) di curl mostra l'intera conversazione HTTP. Avviare prima il backend:

```
# Terminale 1: avvia il backend
python http_backend.py 8001
```

Poi, in un secondo terminale:

```
# Terminale 2: richiesta verbose
curl -v http://127.0.0.1:8001/
```

Output atteso (semplificato):

```
> GET / HTTP/1.1
> Host: 127.0.0.1:8001
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.0 200 OK
< Content-Type: text/html; charset=utf-8
< Content-Length: ...
< X-Served-By: backend-8001
< Cache-Control: no-store
<
<!DOCTYPE html>...
```

Esercizio 2.1

Provare i seguenti comandi e annotare le differenze nella risposta:

`curl -v http://127.0.0.1:8001/api/info`

`curl -v http://127.0.0.1:8001/api/echo?nome=Mario&corso=ASD`

`curl -v http://127.0.0.1:8001/health`

`curl -v http://127.0.0.1:8001/pagina-inesistente`

Domande: Cosa cambia tra gli endpoint? Quale Content-Type viene restituito? Quale status code per l'ultimo?

2.2 HTTP "al rallentatore" con netcat

Per capire che HTTP è davvero solo testo su TCP, possiamo comporre la richiesta a mano:

```
# Connessione raw al server
nc 127.0.0.1 8001
```

```
# Digitare esattamente (poi Invio due volte):
GET /api/info HTTP/1.1
Host: 127.0.0.1:8001
```

Netcat invierà i byte così come li digitiamo. Il server risponderà con la status line, gli header e il body JSON — dimostrando che non c'è nessuna magia: HTTP è testo strutturato su una connessione TCP.

Parte 3 — Analisi del Codice: http_backend.py

3.1 Architettura del Server

Il backend è costruito sulla classe `http.server.BaseHTTPRequestHandler` della libreria standard Python. Questo approccio, volutamente a basso livello, permette di osservare la meccanica del protocollo senza l'astrazione di un framework.

Componenti principali:

- `BackendHandler` — gestisce le richieste implementando `do_GET()`. Ogni endpoint è una funzione privata.
- `create_handler_class(port)` — factory function che inietta la porta come variabile di classe. Pattern necessario perché `HTTPServer` istanzia l'handler per ogni richiesta.
- `HTTPServer` — il server TCP sottostante (dalla libreria standard). Accetta connessioni e delega all'handler.

3.2 La Mappa degli Endpoint

Endpoint	Metodo	Content-Type	Descrizione
`/`	GET	text/html	Pagina HTML con identità del server, header Host, timestamp
`/api/info`	GET	application/json	Metadati JSON: porta, Host header, timestamp, client IP
`/api/echo`	GET	application/json	Echo dei query parameters (utile per test di redirect)
`/health`	GET	application/json	Health check: {"status": "healthy", "port": ...}
Qualsiasi altro	GET	text/plain	404 Not Found

3.3 Il Metodo _send_response()

Tutte le risposte passano da un unico metodo centralizzato che assembla i quattro componenti di una risposta HTTP:

7. **Status Line:** generata da `self.send_response(status)` — produce es. `HTTP/1.0 200 OK`
8. **Headers:** aggiunti con `self.send_header(nome, valore)` — Content-Type, Content-Length, e gli header custom
9. **Separatore:** `self.end_headers()` — invia la riga vuota \r\n che separa headers e body
10. **Body:** `self.wfile.write(body)` — scrive i byte del contenuto sulla socket

Notiamo due header significativi dal punto di vista architetturale:

- `X-Served-By: backend-8001` — header custom per il debugging del load balancing, analogo a quelli usati da CDN come Fastly e Varnish in produzione
- `Cache-Control: no-store` — impedisce al browser di cachare la risposta, fondamentale per osservare correttamente il routing del load balancer

3.4 Parsing dell'URL

Il metodo `do_GET()` usa `urlparse()` per separare l'URL nei suoi componenti:

```
parsed = urlparse(self.path)
path  = parsed.path      # es. "/api/echo"
query = parse_qs(parsed.query)  # es. {"nome": ["Mario"]}
```

La funzione `parse_qs()` trasforma la query string in un dizionario dove ogni chiave mappa a una lista di valori (poiché un parametro può apparire più volte: `?tag=a&tag=b`).

Esercizio 3.1 — Estendere il Backend

Aggiungere un nuovo endpoint `GET /api/time` che restituisca in JSON l'ora corrente e il fuso orario del server. Testare con curl e verificare il Content-Type nella risposta.

Parte 4 — HTTP Redirect Load Balancing

4.1 Il Concetto di Redirect

Un redirect HTTP è un meccanismo in cui il server risponde alla richiesta del client non con il contenuto richiesto, ma con un'istruzione: "la risorsa che cerchi si trova altrove". Il client (tipicamente un browser) segue automaticamente questa istruzione.

La risposta redirect contiene:

- Uno status code 3xx (301, 302, 307, 308)
- L'header `Location` con l'URL di destinazione
- Un body opzionale (HTML di fallback per client che non seguono il redirect)

4.2 Il Flusso Completo

Il load balancer basato su redirect opera in questo modo:

Sequenza di una richiesta con redirect

```
1. Client  ─────── GET / ──────────────────────→  LB (:8080)

2. LB sceglie backend (round-robin) → :8001

3. LB  ──────── 302 Found ──────────────────────→  Client
            Location: http://127.0.0.1:8001/

4. Client  ─────── GET / ──────────────────────→  Backend :8001

5. Backend ─────── 200 OK (HTML) ──────────────→  Client
```

A differenza di un reverse proxy (L4 o L7), qui il load balancer **non trasferisce mai il contenuto**. Il client parla direttamente con il backend dopo il redirect. Questo ha conseguenze importanti:

- **Pro:** il LB non è un collo di bottiglia per il traffico dati
- **Pro:** implementazione estremamente semplice
- **Contro:** i backend devono essere raggiungibili direttamente dal client
- **Contro:** il client vede cambiare l'URL nella barra del browser

4.3 301 vs 302 — La Differenza Critica

La scelta tra redirect temporaneo e permanente ha implicazioni architetturali profonde:

Aspetto	302 Found (Temporary)	301 Moved Permanently
Caching	Il browser NON memorizza il redirect	Il browser MEMORIZZA il redirect nella cache
Flusso	Ogni richiesta passa dal LB	Solo la prima richiesta passa dal LB
Controllo del LB	Pieno: decide ad ogni richiesta	Perso: il browser bypassa il LB
Overhead di rete	Doppia connessione per ogni richiesta	Una sola connessione dopo la prima
Failover	LB può escludere backend guasti	Client continua a puntare al backend (anche se guasto)
Analogia	Simile a un proxy dinamico	Simile a DNS-based load balancing

Attenzione Critica — 301 e Cache

Con 301, il browser memorizza il redirect in modo permanente. Se il backend cade, il client continuerà a tentare di raggiungerlo **senza tornare al load balancer**. Per resettare, l'utente deve svuotare la cache del browser. In produzione, 301 è usato per scenari come la migrazione permanente di dominio (es. http → https), non per il load balancing dinamico.

Parte 5 — Analisi del Codice: http_redirect.py

5.1 Componenti del Load Balancer

Il file è organizzato in tre componenti principali:

1. RoundRobinBalancer — Seleziona i backend in modo ciclico e thread-safe.

```
class RoundRobinBalancer:
    def __init__(self, backends):
        self._cycle = cycle(backends)  # itertools.cycle
        self._lock = threading.Lock()  # thread-safety
        self._counter = 0

    def next_backend(self):
        with self._lock:
            backend = next(self._cycle)
            self._counter += 1
            return backend
```

Il lock è necessario perché `HTTPServer` può gestire richieste concorrenti. Senza il lock, due thread potrebbero leggere lo stesso backend dallo stesso stato del cycle.

2. RedirectHandler — Il cuore del load balancer. Ecco il flusso di `do_GET()`:

11. Se il path è `/status` o `/stats`, serve direttamente (non redirige)
12. Altrimenti, chiede al balancer il prossimo backend
13. Costruisce l'URL di destinazione preservando il path originale
14. Invia la risposta redirect con l'header `Location`

```
host, port = self.balancer.next_backend()
target_url = f"http://{host}:{port}{self.path}"

self.send_response(self.redirect_code)  # 301 o 302
self.send_header("Location", target_url)
```

3. Header diagnostici — Il LB aggiunge tre header custom per il debugging:

- `X-LB-Mode: redirect-302` — indica la modalità operativa
- `X-LB-Backend: 127.0.0.1:8001` — il backend selezionato
- `X-LB-Request-Count: 5` — contatore globale di richieste

5.2 La Logica di Caching per 301 vs 302

Il codice gestisce il caching in modo diverso per le due modalità:

- **302:** aggiunge `Cache-Control: no-store, no-cache` per precauzione, anche se 302 non viene cachato di default
- **301:** non aggiunge **nessun** Cache-Control, lasciando che il browser cachi il redirect come previsto dalla specifica HTTP. Questo è intenzionale: permette allo studente di osservare il comportamento di default del browser

5.3 Il Factory Pattern per l'Iniezione

Sia il backend che il LB usano lo stesso pattern per iniettare configurazione nell'handler:

```
def create_redirect_handler(balancer, redirect_code):
    class Handler(RedirectHandler):
        pass
    Handler.balancer = balancer
    Handler.redirect_code = redirect_code
    return Handler
```

Questo pattern è necessario perché `HTTPServer` istanzia l'handler internamente per ogni richiesta, senza permettere di passare argomenti al costruttore. La soluzione è creare una sottoclasse con le variabili di classe impostate.

Parte 6 — Esercitazione Guidata

6.1 Setup dell'Ambiente

Aprire tre terminali e avviare i componenti in quest'ordine:

```
# Terminale 1 — Backend A
python http_backend.py 8001

# Terminale 2 — Backend B
python http_backend.py 8002

# Terminale 3 — Load Balancer (modalità 302)
python http_redirect.py --mode 302
```

6.2 Esperimento 1: Redirect 302 con curl

Osserviamo il redirect senza seguirlo:

```
curl -v http://127.0.0.1:8080/
```

Cosa osservare: lo status code 302, l'header `Location` che punta a uno dei due backend, e il body HTML di fallback.

Ora seguiamo il redirect con il flag `-L`:

```
curl -v -L http://127.0.0.1:8080/
```

Cosa osservare: curl mostra due scambi HTTP completi — il primo verso il LB (302), il secondo verso il backend (200).

Ripetiamo per osservare il round-robin:

```
curl -s -L http://127.0.0.1:8080/api/info | python -m json.tool
curl -s -L http://127.0.0.1:8080/api/info | python -m json.tool
curl -s -L http://127.0.0.1:8080/api/info | python -m json.tool
```

Esercizio 6.2 — Verificare il Round-Robin

Eseguire 10 richieste consecutive e annotare la porta del backend che risponde. Il pattern è alternato (8001, 8002, 8001, 8002...)? Scrivere un breve script bash che automatizzi il test.

6.3 Esperimento 2: 302 nel Browser

Aprire Chrome e navigare a:

http://127.0.0.1:8080/

Aprire i Developer Tools (F12) e selezionare il tab **Network**. Ricaricare la pagina con Ctrl+R.

Cosa osservare nel Network tab:

15. La prima richiesta verso `:8080` con status **302**
16. La seconda richiesta (automatica) verso il backend con status **200**
17. L'header `Location` nella risposta 302
18. Il cambio dell'URL nella barra degli indirizzi

Ogni volta che si ricarica, il contatore sulla pagina di status (`http://127.0.0.1:8080/status`) aumenta, confermando che ogni richiesta passa dal load balancer.

6.4 Esperimento 3: Il Dramma del 301

Fermare il load balancer (Ctrl+C) e riavviarlo in modalità 301:

```
python http_redirect.py --mode 301
```

In una finestra di navigazione privata (importante!), navigare a:

http://127.0.0.1:8080/

Verificare nel Network tab che appare il 301. Ora ricaricare più volte la pagina.

> **Domanda Critica**
> **Osservazione:** dopo il primo caricamento, la pagina di status (`/status`) mostra che il contatore resta fermo. Le richieste successive non passano più dal load balancer. Perché? Cosa succede se il backend scelto viene spento?

Provare a spegnere il backend scelto dal browser (es. il backend su porta 8001). Ricaricare la pagina. Il browser mostrerà un errore di connessione — perché continua a puntare al backend memorizzato nella sua cache, anche se non esiste più.

Per resettare: svuotare la cache del browser (Ctrl+Shift+Del → Immagini e file nella cache) oppure chiudere la finestra privata e aprirne una nuova.

Parte 7 — Esercizi Proposti

Esercizio 1 — Script di Test Automatizzato

Scrivere uno script Python che invii N richieste al load balancer (con `requests` o `urllib`) in modalità 302 e stampi la distribuzione delle risposte tra i backend. Suggerimento: usare l'endpoint `/api/info` e parsare il JSON per estrarre `server_port`.

Esercizio 2 — Aggiungere un Backend Dinamicamente

Modificare `http_redirect.py` per aggiungere un endpoint `POST /admin/add-backend` che accetti un body JSON con host e porta di un nuovo backend e lo aggiunga al pool. Come gestire la thread-safety?

Esercizio 3 — Health Check e Failover

Implementare un thread di background che ogni 5 secondi invii una richiesta a `/health` di ciascun backend. Se un backend non risponde, rimuoverlo temporaneamente dal pool di round-robin. Se torna online, reinserirlo.

Esercizio 4 — Confronto Quantitativo 301 vs 302

Usando gli strumenti sviluppatore del browser, misurare i tempi di caricamento della pagina con 302 e con 301 (dopo il primo redirect). Annotare TTFB (Time To First Byte) e il numero totale di connessioni. Riportare i risultati in una tabella.

Esercizio 5 — Da Redirect a Reverse Proxy (Challenge)

Confrontare questo approccio a redirect con il load balancer TCP visto nella lezione precedente (`labs/lb_example/tcp/`). Scrivere un breve documento (max 1 pagina) che confronti i due approcci su: trasparenza per il client, overhead di rete, complessità di implementazione, tolleranza ai guasti.

Parte 8 — Riepilogo

Concetto	Cosa abbiamo visto
Protocollo HTTP	Modello request/response, struttura dei messaggi, metodi, status code, header
Status Code 3xx	Meccanismo di redirect: il server indica dove trovare la risorsa
302 vs 301	Temporaneo (il LB mantiene il controllo) vs permanente (il browser bypassa il LB)
Load Balancing L7	Distribuzione del traffico a livello applicativo tramite redirect
Round-Robin	Strategia ciclica di selezione del backend (con thread-safety)
Header Custom	X-Served-By, X-LB-Mode per il debugging in scenari distribuiti
Cache del Browser	Impatto del caching su architetture distribuite (301 + cache = loss of control)

Riferimenti

- RFC 7230–7235 — HTTP/1.1 (semantica, routing, caching, autenticazione)
- RFC 9110 — HTTP Semantics (consolidamento delle RFC precedenti)
- MDN Web Docs — HTTP: developer.mozilla.org/en-US/docs/Web/HTTP
- Documentazione Python — http.server: docs.python.org/3/library/http.server.html
- "Web Scalability for Startup Engineers", Ejsmont — Cap. 3: Load Balancing

— Fine del Handout —